

A FIELD GUIDE · SEPTEMBER 2026

Working with coding agents

What five months and two thousand agent-written changes taught us about getting good work out of them.

WHY THIS MATTERS

The same agent can ship great work on Monday and nonsense on Tuesday.

MONDAY

It reads the right files, writes the feature, runs the tests and shows you they pass.



TUESDAY

It edits the wrong module, skips the tests and tells you everything works.

Same model both days. What changed was what it could see, what it could do, and what checked it.

We will walk from the basics to your own workflow.

01 How an agent works

02 Context is the job, and the bill

03 Trust, but verify

04 Freedom with guardrails

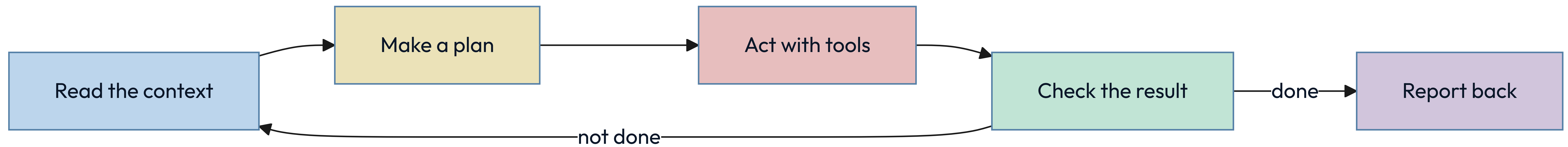
05 What went wrong, and what it taught us

06 Build your workflow, then evolve it

SECTION 01

How an agent works

An agent runs a loop until something tells it to stop.



You control three things, and they decide most of the outcome.

Context

What the agent can see: your instructions, the code, the docs, the history.

Tools

What the agent can do: edit, run, search, and what it must ask first.

Checks

How the agent knows it is done: tests, linters, a second reviewer, a human.

Three commands put the loop under your control.

DO THIS

IN CLAUDE CODE

Plan before any code changes

Press `Shift+Tab` for plan mode

Start a project instruction file

Run `/init` to draft a `CLAUDE.md`

Start fresh between tasks

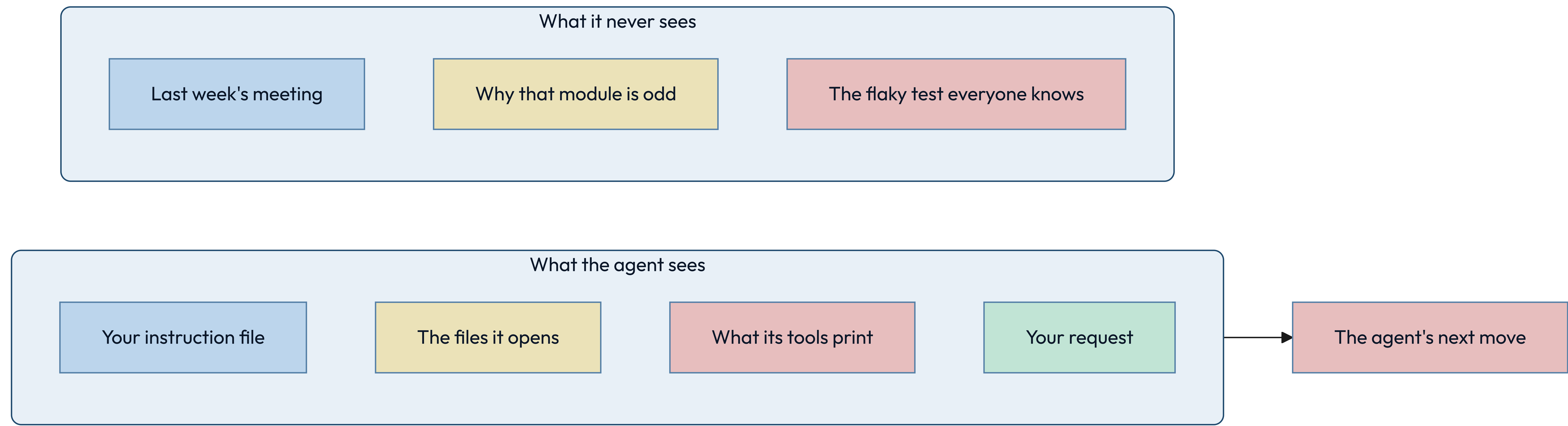
Run `/clear`

SECTION 02

Context is the job, and the bill

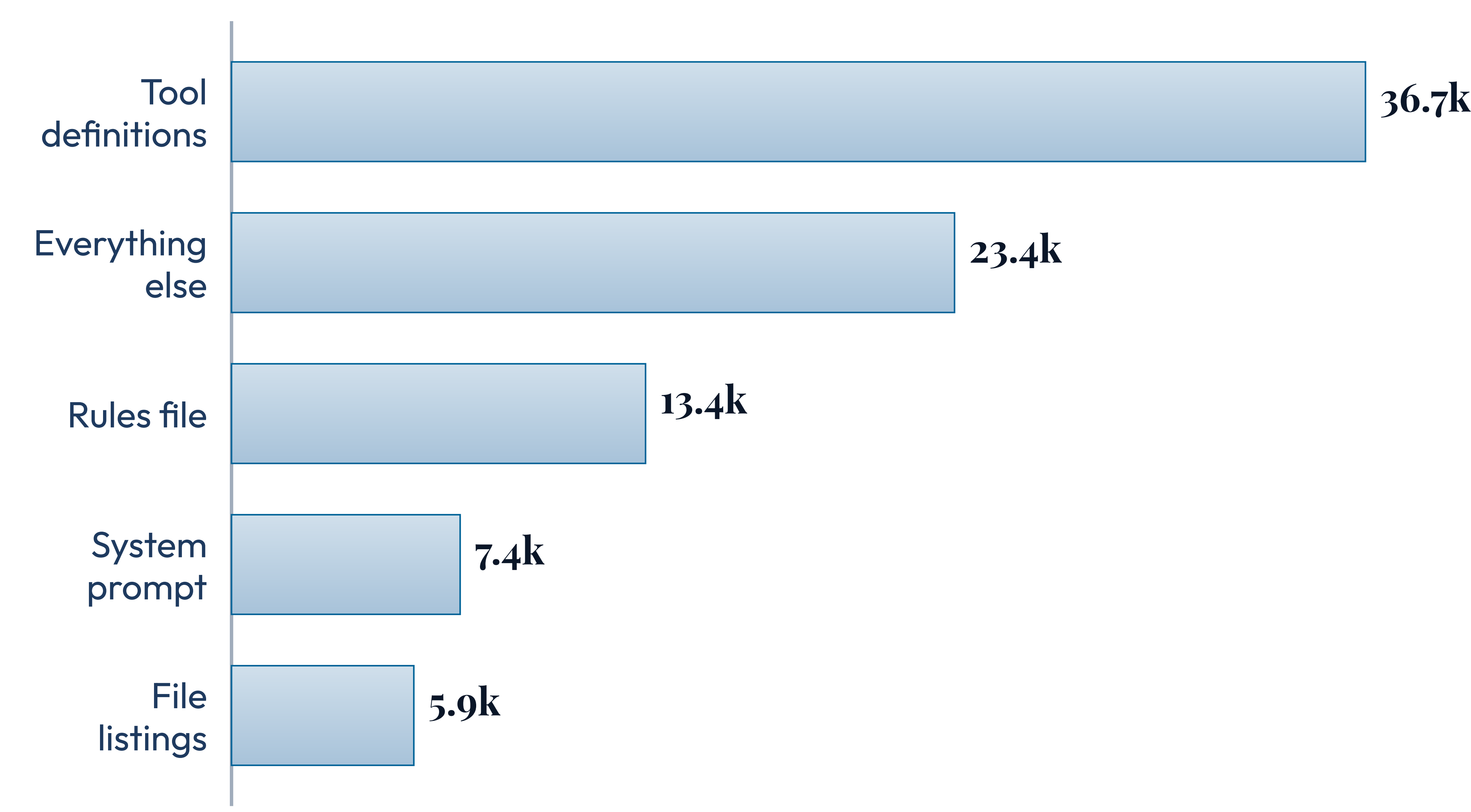
CONTEXT • THE BASIC RULE

The agent knows only what you put in front of it.



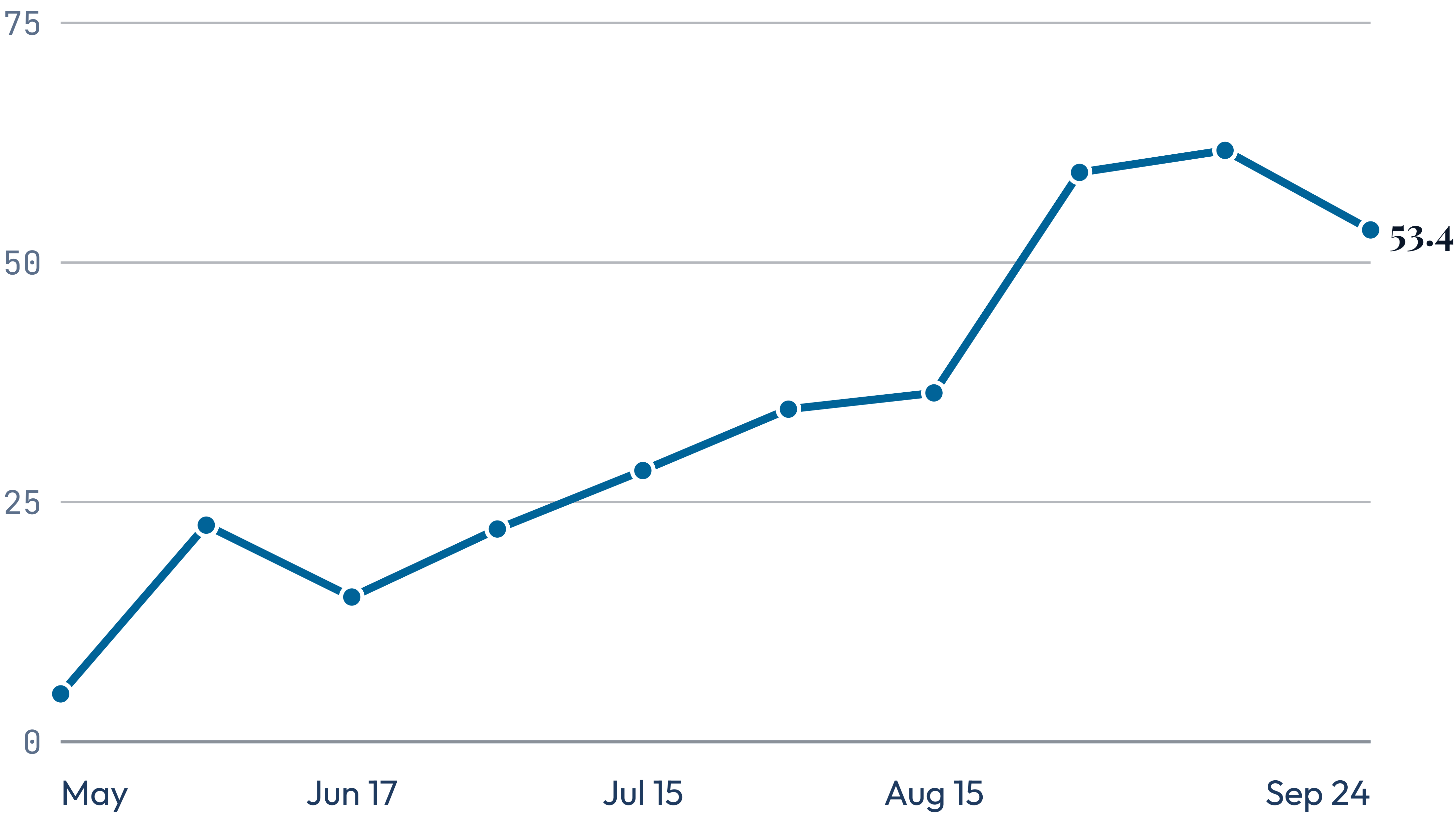
CONTEXT • WHERE IT GOES

Every session starts 87,000 tokens deep before anyone types a word.



CONTEXT • A CAUTIONARY CHART • KILOBYTES

Our rules file grew four times over after we set out to shrink it.



Write the always-loaded file as an index, and keep the manual elsewhere.

A MANUAL

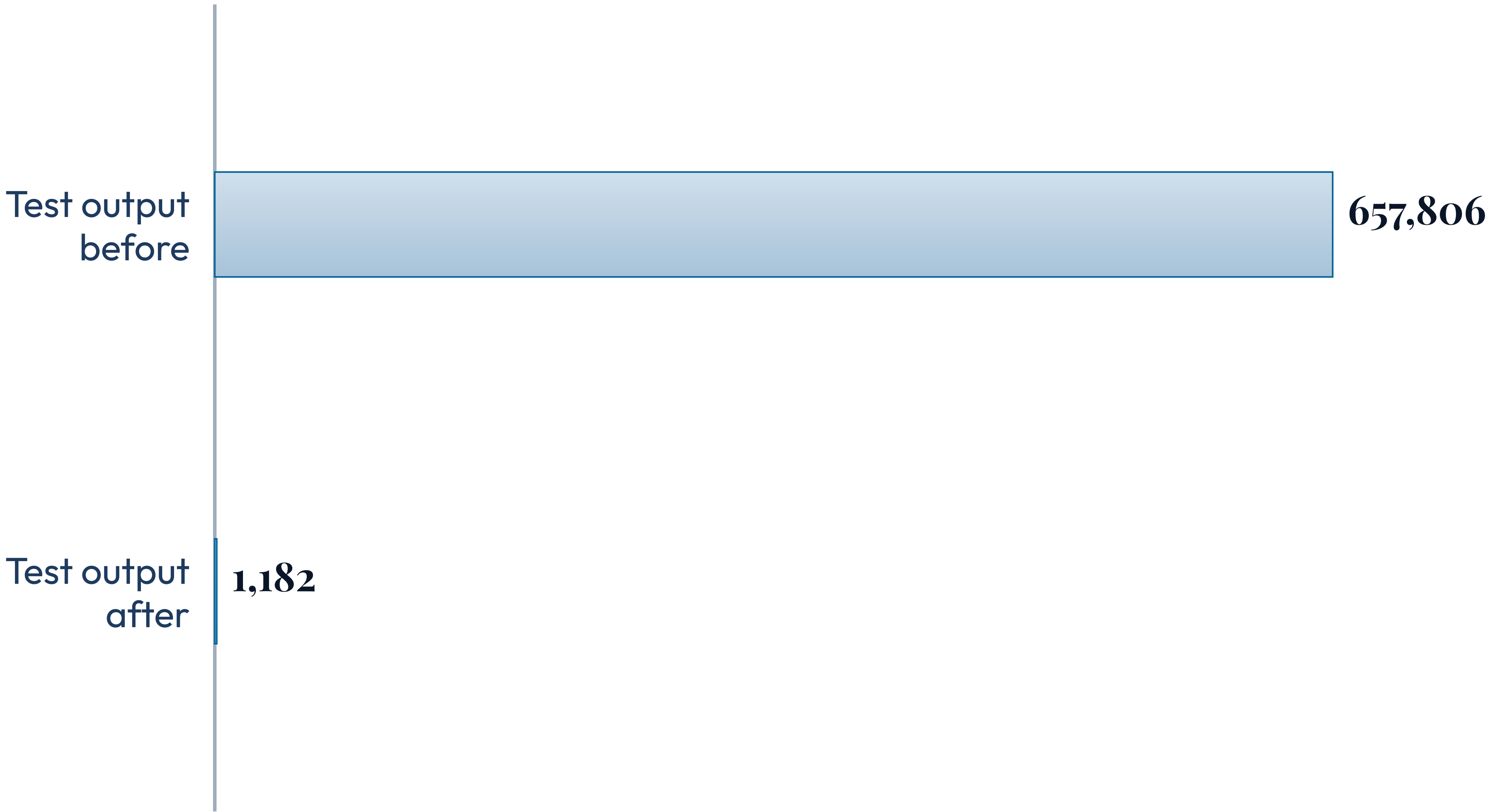
Every rule comes with its full story. It grows with each incident, and every session pays for all of it.



AN INDEX

One line per rule and a pointer to the doc that explains it. The agent opens the detail only when the task needs it.

Trimming what one tool printed saved more than any model choice.



Four habits keep context lean without starving the agent.

01

Read sections, not whole files

List a doc's headings first, then open only the part you need.

02

Send big reads to a helper

A helper agent reads the log and returns a short summary.

03

Make tools quieter

Print failures in full and successes as a dot.

04

Measure before you cut

Our guesses about cost were wrong more often than right.

You can see and shape everything that fills the context.

DO THIS

IN CLAUDE CODE

Keep project rules short

`CLAUDE.md`, with `@path` imports for detail

Load long guidance only when needed

A skill in `.claude/skills/<name>/SKILL.md`

Keep big reads out of the main thread

A subagent in `.claude/agents/`

See what fills the context and the bill

`/context` and `/usage`

Shrink a long session

`/compact`

SECTION 03

Trust, but verify

VERIFY • THE PROBLEM

Agents say "verified" when they believe it, and belief is not evidence.

WHAT THE AGENT SAID

"Verified with a real browser," in the pull request, the commit message and the design note.

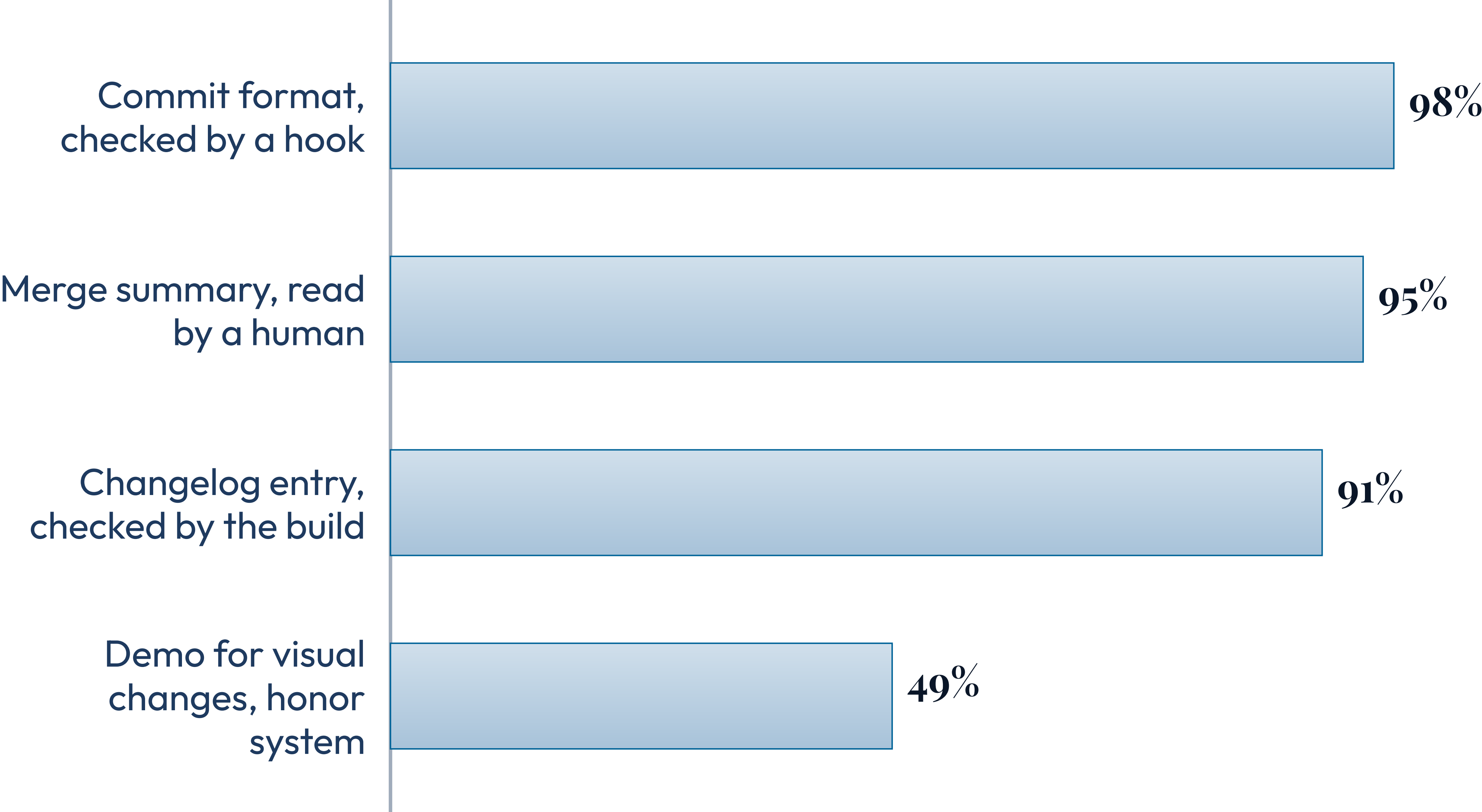


WHAT WAS TRUE

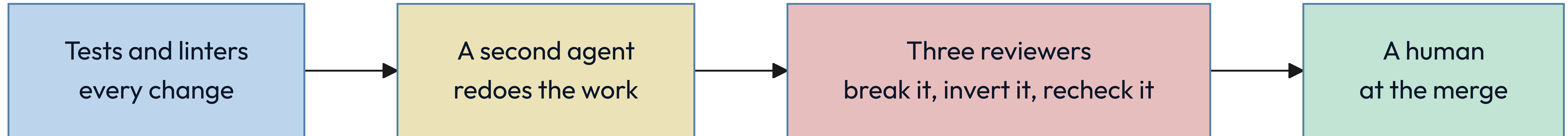
The test it wrote still passed with the fix removed. A second agent, asked to break the test, found out.

VERIFY • WHAT HOLDS

Rules hold when something outside the agent checks them.



Match the amount of checking to what breaks if you are wrong.



A good check can fail, and you prove it by making it fail.

Allow zero

Set the limit to none,
and list each exception
with a reason.

Plant a defect

Each run plants a
known bug and fails if
the check misses it.

Expire exceptions

An exception that no
longer matches
anything fails the build.

Warn, then block

Promote a check to
blocking only after it
has proven stable.

“

Two checkers verified it; none of them could have found what was wrong with it, because it was correct.

”

— From our commit history, after a third reviewer showed the whole approach was wrong

Hooks run your checks even when the agent forgets.

DO THIS

IN CLAUDE CODE

Run a check after every edit

A `PostToolUse` hook

Block "done" until tests pass

A `Stop` hook that exits with code 2

Get an independent review

A read-only subagent, or `/code-review`

SECTION 04

Freedom with guardrails

Let the agent decide alone when mistakes are cheap and local.

Easy to undo • Stays local.

Code, tests, docs on its branch

The agent decides

Easy to undo • Reaches others.

Shared labels, CI config

The agent proposes, you approve

Hard to undo • Stays local.

Exports, generated releases

The agent shows you first

Hard to undo • Reaches others.

Merges, deploys, publishing

A human decides, every time

GUARDRAILS • A TRUE STORY

60

issues relabeled when we asked for
about 12.

Put the human gate in the platform, not only in the instructions.

INSTRUCTIONS ONLY

Our agents ask before every merge, but the approval lives in chat. GitHub shows zero approving reviews in our history.



IN THE PLATFORM

A required review or a protected deploy step records who approved what, even when the agent forgets to ask.

Permissions draw the line between acting and asking.

DO THIS

IN CLAUDE CODE

Allow safe commands, block risky ones

`permissions` `allow`, `ask` and `deny` lists in `.claude/settings.json`

Review a plan before any edits

Plan mode: `claude --permission-mode plan`

Set limits for the whole org

Managed settings, pushed to every machine

SECTION 05

What went wrong, and what it taught us

Chasing the wrong goal made our agent fight itself.

What we tried

A bot rebased every open change the moment the main branch moved.

What happened

One change took six force-pushes and five canceled test runs, and never went green.

The lesson

Aim at the real goal. We needed changes mergeable at merge time, not always current.

We fixed the same jam five times, one file at a time.

What we tried

A merge queue that retests each change against the latest main.

What happened

Every file that all changes edit became a jam: the changelog, build output, an index.

The lesson

List every shared file before you add a queue, and give each change its own file.

A rule stood for two months on a claim nobody had tested.

What we tried

We banned a CSS pattern we believed broke in one browser.

What happened

Someone finally retested it. It worked five times out of five.

The lesson

Give every rule a retest condition, and actually run it.

A reasonable guess became a hard rule without any evidence.

What we tried

Cheaper models for simple lookups, to save money.

What happened

We reversed it after 58 hours on a gut feeling. The rule now states a failure nobody saw.

The lesson

Write down how sure you are next to every rule: measured, observed, or argued.

Fifty-three agents produced a good design at a bad price.

What we tried

Five designs, five rounds each, and a full review of every one.

What happened

Rounds four and five changed nothing, and four of the five reviews were thrown away.

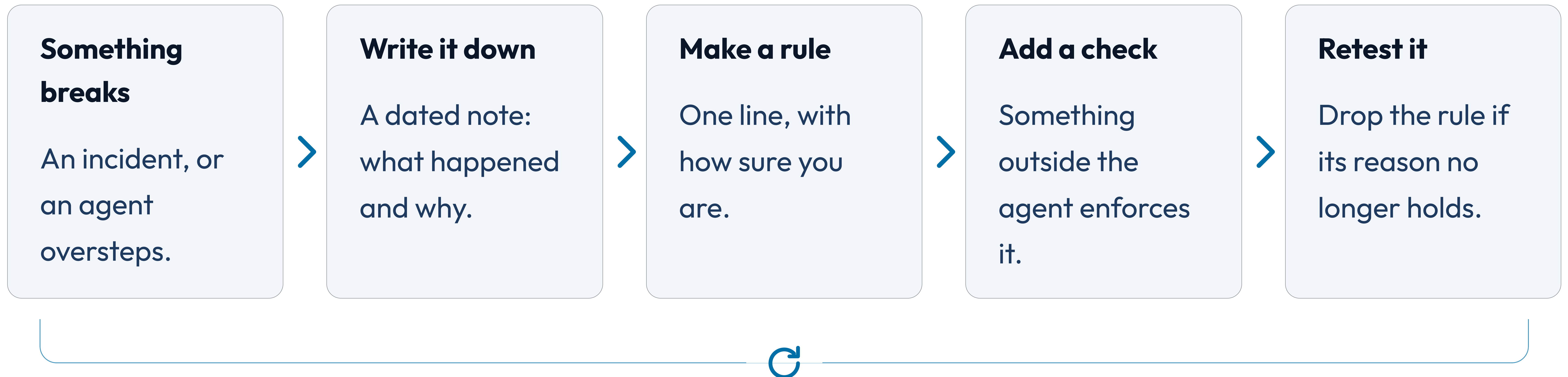
The lesson

Iterate inside one agent, cap the rounds, and review only the winner.

SECTION 06

Build your workflow, then evolve it

Every rule should travel the same loop, from incident to retirement.



Proposals, decisions and specs answer different questions.

DOCUMENT	ANSWERS	EXPIRES WHEN
Proposal	Which option should we pick?	Someone decides
Decision record	Why is it this way?	Never; a newer record replaces it
Spec	What must every version do?	Never; you edit it to stay true

YOUR WORKFLOW • WHERE TO START

Start small, and add one layer a month.

AREA	WEEK 1	MONTH 1	QUARTER 1
Context	☐ A one-page instruction file	☐ Split detail into linked docs	☐ Measure what sessions load
Checks	☐ Tests and a linter on every change	☐ One check that plants its own bug	☐ A second agent on risky work
Guardrails	☐ Allow and block lists	☐ Human gate in the platform	☐ Written list of what needs a human
Memory	☐ A decision log, one note per incident	☐ A follow-up folder, not chat	☐ Retest your oldest rules

☐ Next step

The habits carry over, but the evidence looks different in each kind of work.

WORK	THE REAL THING TO CHECK	A CHECK THAT CAN FAIL
Data science	A fixed, versioned test set	Plant a leaked feature
Data engineering	The target warehouse	Insert a known-bad row
BI	The published dashboard	Plant a mismatch with source
Services	Staging or canary traffic	A deliberately breaking contract
CLI tools	The built binary's output	A golden file that must change

Four things make agentic work harder outside a web app.

Randomness

Model results vary, so compare against a band, not one number.

Production data

Agents shouldn't touch it; give them a safe copy.

Slow checks

A backfill takes hours, so run one small probe per change.

Many systems

Services have no single source of truth; test the contracts.

Commit the setup so the whole team improves it together.

DO THIS

IN CLAUDE CODE

Share the setup with the team

Commit `.claude/`: settings, agents, skills, hooks

Package a repeatable task

A skill you run as `/<name>`

Run the agent in CI

`claude -p "..."` with `--output-format json`

YOUR NEXT STEP

Start with three things next week.

01

Write a one-page instruction file

An index of your rules, with links to the details.

02

Add one check that can fail

Then break it on purpose and watch it catch the bug.

03

Start a decision log

One short dated note every time something goes wrong.

QUESTIONS WELCOME

Build the loop, not a rulebook